



SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL

Trabalho 1 - Sistemas Distribuídos

HENRIQUE ALVES CAMPOS [EF04231]

HENRIQUE DE SOUZA CAMPOS [EF04673]

HERON FILLIPE SILVEIRA SANTOS [EF04211]

Sumário

- Sumário..... 2**
- 1. Introdução..... 3**
 - 1.1. Contextualização..... 3
 - 1.2. Objetivo..... 3
- 2. Arquitetura..... 4**
- 3. Requisitos Funcionais..... 4**
- 4. Casos de Uso..... 5**
- Referências..... 6**

1. Introdução

1.1 Contextualização

Com o avanço das aplicações distribuídas, sistemas que permitem a interação simultânea de múltiplos usuários sobre um mesmo ambiente compartilhado tornaram-se cada vez mais relevantes. Esses sistemas exigem mecanismos eficientes de comunicação, sincronização e gerenciamento de estado, de forma a garantir a consistência das informações mesmo diante de acessos concorrentes.

Nesse contexto, aplicações colaborativas em tempo real têm grande destaque, pois possibilitam que diferentes usuários interajam de maneira dinâmica sobre um mesmo recurso. Exemplos desse tipo de sistema incluem editores colaborativos, quadros interativos e ambientes digitais compartilhados, os quais demandam uma arquitetura capaz de lidar com múltiplas requisições simultâneas e propagação de atualizações entre os participantes.

Este trabalho está inserido nesse cenário ao propor a modelagem de um sistema distribuído denominado *Papel-it*, que simula um mural digital de post-its compartilhado entre múltiplos usuários conectados.

1.2. Objetivo

O objetivo deste trabalho é especificar e modelar o sistema distribuído *Papel-it*, um ambiente colaborativo no qual múltiplos usuários podem interagir simultaneamente em um mural digital estruturado como uma matriz de coordenadas.

O sistema permitirá que usuários conectados realizem operações como criação, edição e remoção de post-its em diferentes posições do mural, além de visualizar, em tempo real, as alterações realizadas por outros usuários. O servidor será responsável por gerenciar o estado global do sistema, validar as operações realizadas e garantir a consistência das informações distribuídas entre os clientes.

Nesta primeira etapa, o foco está na definição dos requisitos funcionais, na modelagem dos casos de uso e na representação do comportamento do sistema por meio de diagramas, estabelecendo uma base sólida para as etapas posteriores de desenvolvimento.

2. Arquitetura

O sistema Papel-it segue o modelo cliente-servidor, no qual um servidor central é responsável por manter o estado global do sistema, representado por uma matriz de post-its. Onde os clientes se conectam ao servidor e enviam requisições, sendo elas, para a criação, edição e remoção de papel-its. O servidor, tem a função de processar essas requisições, validar as operações e atualizar o estado global.

Após cada alteração, o servidor atualiza para todos os clientes conectados, garantindo uma sincronização do sistema. O servidor, também, tem o papel de autoridade central, garantindo consistência dos dados e tratando acessos concorrentes, evitando conflitos na modificação de uma mesma posição da matriz.

3. Requisitos Funcionais

Abaixo, citamos todos os requisitos funcionais do sistema:

RF01 - O sistema deve permitir que o usuário se conecte ao servidor

RF02 - O sistema deve permitir que o usuário informe uma identificação ao se conectar.

RF03 - O sistema deve permitir que o usuário visualize o estado atual da matriz de post-its.

RF04 - O sistema deve enviar ao usuário o estado atual do mural no momento da conexão.

RF05 - O sistema deve permitir que o usuário adicione um novo post-it em uma coordenada disponível.

RF06 - O sistema deve permitir que o usuário edite o conteúdo de um post-it existente.

RF07 - O sistema deve permitir que o usuário remova um post-it existente.

RF08 - O sistema deve garantir que apenas um post-it ocupe cada posição da matriz, impedindo inserções em posições já ocupadas.

RF09 - O sistema deve notificar todos os clientes conectados sempre que houver

alteração no estado do quadro.

RF10 - O sistema deve garantir que todos os clientes mantenham uma visão atualizada e consistente do mural.

RF11 - O sistema deve associar cada post-it ao usuário responsável por sua criação ou edição.

RF12 - O sistema deve registrar o horário da última modificação de cada post-it.

RF13 - O sistema deve informar ao usuário o sucesso ou falha das operações realizadas, incluindo mensagens de erro em casos como tentativa de inserção em posição ocupada.

4. Casos de Uso

UC01 - Conectar ao Sistema

Ator: Usuário

Descrição: Permitir que o usuário se conecte ao sistema e receba o estado inicial do mural.

Requisitos relacionados: RF01, RF02, RF04

Pré-condições:

- O servidor deve estar ativo.

Pós-condições:

- O usuário está conectado.
- O estado do mural foi sincronizado com o cliente.

Fluxo Principal:

1. O usuário informa sua identificação.
2. O cliente envia a requisição de conexão ao servidor.
3. O servidor registra o usuário.
4. O servidor envia o estado atual da matriz ao cliente.
5. O sistema exibe o mural ao usuário.

Fluxo Alternativo:

- 2a. Falha na conexão: o sistema informa erro ao usuário.

UC02 - Visualizar Mural

Ator: Usuário

Descrição: Permitir que o usuário visualize o mural atualizado.

Requisitos relacionados: RF03, RF10

Pré-condições:

- Usuário conectado.

Pós-condições:

- O usuário visualiza o estado atual do mural.

Fluxo Principal:

1. O usuário acessa o sistema.
2. O cliente apresenta o estado atual da matriz.
3. O sistema mantém a visualização sincronizada com o servidor.

UC03 - Criar Papel-it

Ator: Usuário

Descrição: Permitir a criação de um novo papel-it no mural.

Requisitos relacionados: RF05, RF08, RF09, RF10, RF11, RF12, RF13

Pré-condições:

- Usuário conectado.
- Coordenada válida na matriz.

Pós-condições:

- O papel-it é criado e visível para todos os usuários.

Fluxo Principal:

1. O usuário seleciona uma posição (x, y).
2. O usuário insere o conteúdo do papel-it.
3. O cliente envia a requisição ao servidor.
4. O servidor verifica se a posição está livre.
5. O servidor cria o papel-it, associando ao usuário.
6. O servidor registra o horário da criação.
7. O servidor atualiza o estado global da matriz.
8. O servidor envia confirmação de sucesso ao cliente.
9. O servidor notifica todos os clientes.
10. Os clientes atualizam o mural.

Fluxo Alternativo:

- 4a. Posição ocupada no mural, o servidor retorna erro ao cliente.

UC04 - Editar Papel-it

Ator: Usuário

Descrição: Permitir a edição de um papel-it existente.

Requisitos relacionados: RF06, RF09, RF10, RF11, RF12, RF13

Pré-condições:

- Usuário conectado.
- Papel-it existente.

Pós-condições:

- O conteúdo do papel-it é atualizado.

Fluxo Principal:

1. O usuário seleciona um papel-it.
2. O usuário altera o conteúdo.
3. O cliente envia a requisição ao servidor.
4. O servidor valida a existência do papel-it.
5. O servidor atualiza o conteúdo.
6. O servidor registra o horário da modificação.
7. O servidor envia confirmação ao cliente.
8. O servidor notifica todos os clientes.
9. Os clientes atualizam o mural.

Fluxo Alternativo:

- 4a. Papel-it inexistente no mural, o servidor retorna erro.
- 5a. Conflito de edição simultânea, o servidor aplica política de resolução .

UC05 - Remover Papel-it

Ator: Usuário

Descrição: Permitir a remoção de um papel-it.

Requisitos relacionados: RF07, RF09, RF10, RF13

Pré-condições:

- Usuário conectado.
- Papel-it existente.

Pós-condições:

- O papel-it é removido do mural.

Fluxo Principal:

1. O usuário seleciona um papel-it.
2. O usuário solicita remoção.
3. O cliente envia a requisição ao servidor.
4. O servidor valida a existência do papel-it.
5. O servidor remove o papel-it da matriz.
6. O servidor envia confirmação ao cliente.
7. O servidor notifica todos os clientes.
8. Os clientes atualizam o mural.

Fluxo Alternativo:

- 4a. Papel-it inexistente no mural, o servidor retorna erro.

UC06 - Receber Atualizações

Ator: Usuário

Descrição: Manter o mural sincronizado com alterações feitas por outros usuários.

Requisitos relacionados: RF09, RF10

Pré-condições:

- Usuário conectado.

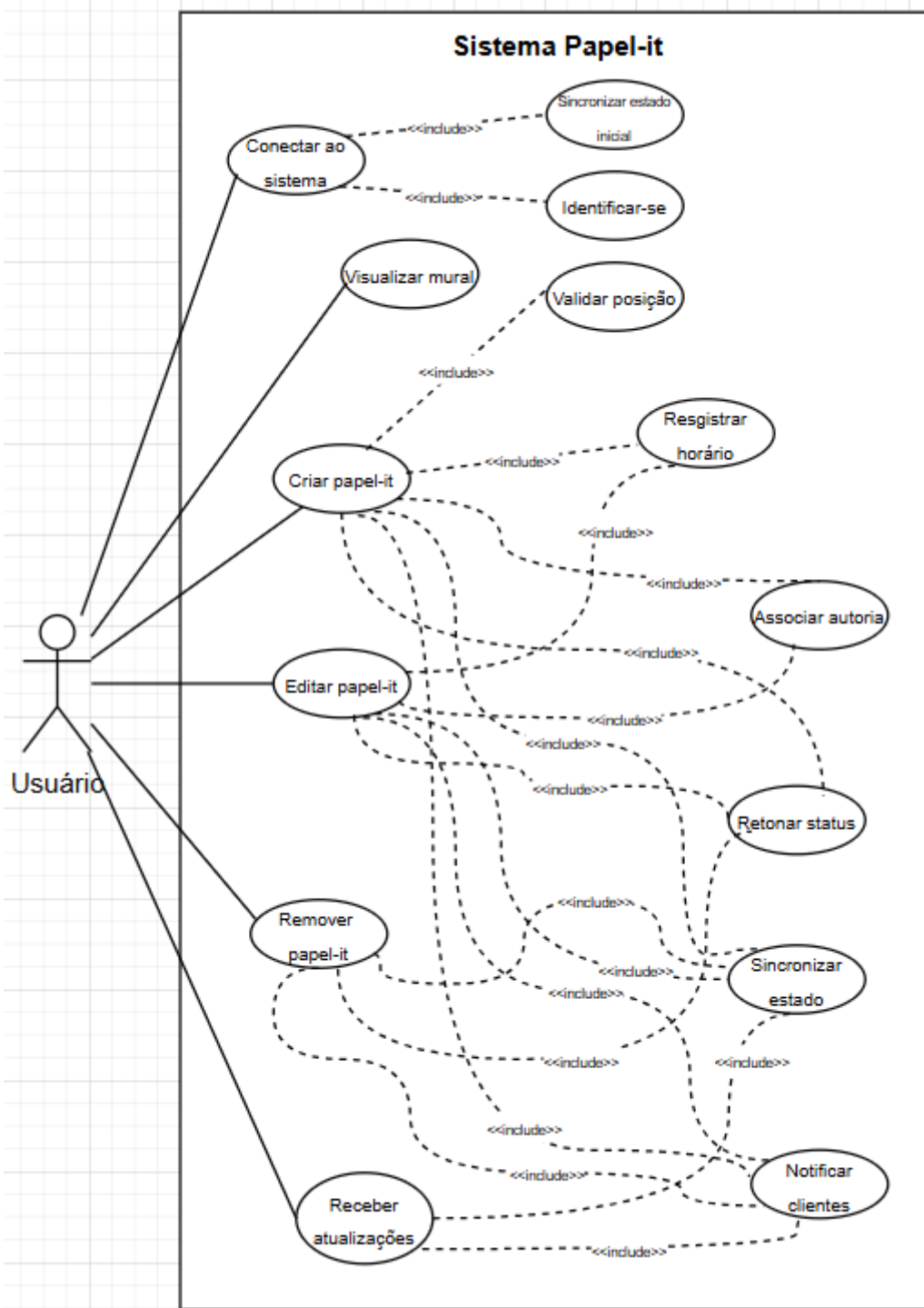
Pós-condições:

- O mural do cliente permanece atualizado.

Fluxo Principal:

1. O servidor detecta uma alteração no mural.
2. O servidor envia uma notificação aos clientes.
3. O cliente recebe a atualização.
4. O cliente aplica a alteração localmente.
5. O sistema atualiza a visualização.

4.1. Diagrama UML



Referências

[1] TANENBAUM, Andrew S.; VAN STEEN, Maarten. Sistemas Distribuídos: Princípios e Paradigmas. 2. ed. São Paulo: Pearson, 2007.

[2] PYTHON SOFTWARE FOUNDATION. Python Documentation – socket —

Low-level networking interface. Disponível em:
<https://docs.python.org/3/library/socket.html>. Acesso em: 2026.

[3] PYTHON SOFTWARE FOUNDATION. Python Documentation – threading — Thread-based parallelism. Disponível em:
<https://docs.python.org/3/library/threading.html>. Acesso em: 2026.

[4] PYTHON SOFTWARE FOUNDATION. Python Documentation – tkinter — Python interface to Tcl/Tk. Disponível em: <https://docs.python.org/3/library/tkinter.html>. Acesso em: 2026.

[5] COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. Distributed Systems: Concepts and Design. 5. ed. Addison-Wesley, 2011.

[6] FIELDING, Roy T. Architectural Styles and the Design of Network-based Software Architectures (REST). 2000.

[7] GUEDES, Gilleanes T. A. UML 2: Guia Prático. São Paulo: Novatec, 2011.